

oneDAL HDBSCAN: performance, correctness and optimization

Scope: `cpp/daal/src/algorithms/hdbscan` (CPU) and `cpp/oneapi/dal/algo/hdbscan` (oneAPI / GPU), as integrated into `scikit-learn-intelex`

Related PRs: [oneDAL #3750](#), [scikit-learn-intelex #3124](#), [scikit-learn_bench #225](#)

Contents

- 1. What was measured and how
- 2. Performance summary
 - 2.1 Against stock scikit-learn
 - 2.2 The brute-force path, before and after the change in this PR
 - 2.3 Where the dense path's time goes
- 3. Correctness and conformance
 - 3.1 Clustering agreement with scikit-learn
 - 3.2 The one case with a large disagreement, and why it is not a bug
 - 3.3 Tie degeneracy is a general property of HDBSCAN, not of this dataset
 - 3.4 scikit-learn's own HDBSCAN test suite, run under patching
- 4. Bugs found
 - 4.1 Serial `finalize` on the shared Euclidean distance primitive (fixed)
 - 4.2 Unstable MST edge sort (fixed)
 - 4.3 Truncated MST handed to the dendrogram builder (fixed)
 - 4.4 `mcsMax` bypassed for leaf clusters (fixed)
 - 4.5 `clusterSelectionEpsilon` can select an ancestor and its descendant (open)
 - 4.6 No single-cluster lambda threshold in labelling (open)
 - 4.7 `kd_tree` far-child pruning is weaker than documented (open, cosmetic)
- 5. Optimizations: what worked, what did not
 - 5.1 Accepted: threaded `finalize`
 - 5.2 Accepted: bounded select-k for core distances
 - 5.3 Rejected: Prim's algorithm instead of Boruvka (20-25% slower)
 - 5.4 Rejected: exact nearest-MRD candidate cache (1.03x, not worth it)
- 6. Remaining optimization paths
 - 6.1 Keep the Euclidean matrix in squared space through the MST build
 - 6.2 Exploit the symmetry of the distance matrix
 - 6.3 The `kd_tree` split-plane far-child guard
 - 6.4 Return the single-linkage tree and the membership probabilities
- 7. Test coverage added

1. What was measured and how

All numbers in this report come from the in-repo `scikit-learn_bench` harness, using the HDBSCAN configurations added in `scikit-learn_bench #225` (`configs/weekly/hdbscan.json` and `configs/experiments/hdbscan_parameters.json`), so every case is reproducible from a config file rather than from a throwaway script.

Host: 2 x Intel Xeon Platinum 8480L, 56 cores / 112 threads per socket, 224 threads total, 2 NUMA nodes, 499 GB RAM

Reference: stock scikit-learn `sklearn.cluster.HDBSCAN`

Under test: `sklearnex.preview.cluster.HDBSCAN`, which dispatches to oneDAL through `onedal.cluster.HDBSCAN`

Metrics reported:

fit time (best of the measured repetitions), number of clusters, Davies-Bouldin score, and homogeneity / completeness against the ground truth where labels exist

90 cases were run, of which 45 pair up between the two libraries (the remainder are cases only one library can run, e.g. metrics oneDAL does not support). Each case fixes dataset, method, metric, shape, dtype, `min_cluster_size`, `min_samples`, `cluster_selection_method`, `store_centers`, data format and memory order, and those eleven fields form the key that before/after runs are merged on.

2. Performance summary

2.1 Against stock scikit-learn

Aggregated over the 45 paired cases:

Method	n	geomean	min	max
<code>kd_tree</code>	23	191.3x	24.4x	475.8x
<code>ball_tree</code>	11	96.3x	37.1x	187.9x
<code>brute</code>	11	4.1x	2.9x	6.8x
overall	45	63.1x	2.9x	475.8x

No case is slower than scikit-learn; 34 of 45 are at least 10x faster.

The spread between the tree methods and brute force is not a property of oneDAL's implementation quality but of what each side does. scikit-learn's tree path builds the mutual reachability MST with Boruvka over a single `cython`-level tree and does not thread the traversal, while oneDAL's tree kernels thread the per-point nearest-different-component query. Brute force, by contrast, is dominated on both sides by dense linear algebra that is already threaded and vectorized, so the remaining gap is the algorithmic overhead around it.

Per-method highlights (float64 unless noted):

- `kd_tree`, `make_blobs` 100000 x 4, `min_cluster_size`=5: 48.6 s -> 102 ms (475.8x), identical labels.
- `kd_tree`, `skin_segmentation` 100000 x 3: 47.1 s -> 121 ms (389.1x), 4847 vs 4886 clusters, Davies-Bouldin 1.203 vs 1.209.
- `ball_tree`, `covtype` 50000 x 54: 197.4 s -> 2.50 s (79.0x), 133 vs 133 clusters, homogeneity and completeness equal to four decimals.
- `kd_tree`, `make_blobs` 100000 x 8 with `minkowski`: 755.3 s -> 4.57 s (165.3x).

2.2 The brute-force path, before and after the change in this PR

The brute-force numbers above were the weakest part of the picture, so they were profiled further. Section 4.1 explains what was found and fixed; the table below is the effect, measured in the same harness with everything else held constant.

Dataset	Metric	n	d	dtype	sklearn	before	after	speedup	vs sklearn
<code>skin_segmentation</code>	euclidean	30000	3	float64	9095.6	2366.4	1186.9	1.99x	3.84x -> 7.66x
<code>skin_segmentation</code>	euclidean	30000	3	float32	8097.6	1347.0	687.0	1.96x	6.01x -> 11.79x
<code>make_blobs</code>	euclidean	25000	64	float64	6286.9	1586.1	819.2	1.94x	3.96x -> 7.67x
<code>sensit</code>	euclidean	25000	100	float64	5427.1	1650.6	868.6	1.90x	3.29x -> 6.25x
<code>make_blobs</code>	euclidean	25000	256	float64	5107.7	1617.7	861.1	1.88x	3.16x -> 5.93x
<code>make_blobs</code>	euclidean	25000	1024	float64	5974.4	1836.7	1023.1	1.80x	3.25x -> 5.84x

Dataset	Metric	n	d	dtype	sklearn	before	after	speedup	vs sklearn
cifar	euclidean	15000	3072	float64	3238.4	1118.1	803.6	1.39x	2.90x -> 4.03x
make_blobs	cosine	25000	64	float64	5393.7	783.1	745.7	1.05x	6.89x -> 7.23x
make_blobs	manhattan	25000	64	float64	5912.1	1025.4	989.1	1.04x	5.77x -> 5.98x
make_blobs	chebyshev	25000	64	float64	5591.4	946.2	941.9	1.00x	5.91x -> 5.94x

All times in milliseconds. Geomean over the ten cases: 1.54x, median 1.84x. Restricted to euclidean, the brute-force geomean against scikit-learn moves from 3.67x to 6.70x.

Two properties of this table are worth stating explicitly, because together they are the evidence that the attribution is right:

- The three non-euclidean cases are flat (1.00x-1.05x). They never enter the code that changed, so they act as a control: the gain is not a measurement artifact or a general improvement in the surrounding code.
- Cluster counts and Davies-Bouldin scores are bit-identical before and after in all ten cases. The change is a pure scheduling change.

2.3 Where the dense path's time goes

Three observations pin the cost of the brute-force path on passes over the $N \times N$ matrix, not on the GEMM that produces it:

1. **The dimension dependence is weak.** At `make_blobs` 25000 rows, raising `d` from 64 to 1024 -- a 16x increase in GEMM work -- added only 217 ms (1586 -> 1837 ms, +14%). If the GEMM dominated, that would have been closer to 16x.
2. **N scaling is clean and quadratic**, consistent with the matrix and the sweeps over it being the cost.
3. **Euclidean was *slower* than manhattan and chebyshev at the same shape** (1586 ms vs 1025 and 946 ms at 25000 x 64), even though euclidean computes its matrix with a single `gemm` while manhattan and chebyshev run an explicit $N^2 \times d$ loop. That inversion can only come from work euclidean does and the others do not.

That third point is what led to the finding in section 4.1: the extra work is the euclidean-specific `finalize` sweep, and the 640 ms gap matches its cost almost exactly.

3. Correctness and conformance

3.1 Clustering agreement with scikit-learn

Across the 45 paired cases, oneDAL and scikit-learn agree on the number of clusters exactly in the large majority, and where they differ the difference is in the last 1% of cluster count with matching quality metrics -- for example `skin_segmentation` 100000 x 3 at 4847 vs 4886 clusters with Davies-Bouldin 1.203 vs 1.209, or `covtype` 50000 x 54 at 133 vs 131.

The two implementations do not agree on label numbering, so all equivalence testing is done on the partition the labels induce (the set of member indices per label), never on the label values.

3.2 The one case with a large disagreement, and why it is not a bug

`skin_segmentation` at 30000 rows, brute force, reports 1728 clusters from scikit-learn against 1562 from oneDAL, with Davies-Bouldin 8.806 vs 1.199. That looks alarming and was investigated as a suspected defect. It is not one:

`skin_segmentation` is uint8 RGB data. In the 30000-row subset there are only 9154 distinct rows, and 17327 of the 30000 points have at least five exact duplicates. With `min_samples=5`, the k-th nearest neighbour of such a point is a copy of itself at distance zero, so its core distance is exactly zero. Every

mutual reachability edge among a duplicate group is then also zero, the MST is not unique, and which spanning tree is produced decides how the condensed tree splits.

The decisive evidence that this is input degeneracy and not an oneDAL defect is that **scikit-learn's own backends disagree with each other on this input**: its `brute` and `kd_tree` paths return different cluster counts for the same data and parameters, for the same reason. oneDAL's answer, with the much better Davies-Bouldin score, is one of the valid ones.

An earlier hypothesis that scikit-learn's brute path had an off-by-one in its k-th neighbour selection was checked and is **wrong**: `_reachability.pyx` uses `further_neighbor_idx = min_samples - 1` for both the sparse and dense branches, which is the same neighbour the tree path's `kneighbors(X, min_samples)` returns.

3.3 Tie degeneracy is a general property of HDBSCAN, not of this dataset

The duplicate-row case is the extreme, but the underlying effect is generic. Every mutual reachability edge whose plain distance is shorter than both endpoints' core distances collapses onto `max(core_i, core_j)`. Ties are therefore dense in any dataset with locally uniform density, and the order in which equal-weight edges are processed decides tie-degenerate splits.

This has a direct consequence for the implementation, addressed in section 4.2: the MST edge sort must be a total order and must be stable, or the labels become a function of the thread schedule.

3.4 scikit-learn's own HDBSCAN test suite, run under patching

`sklearn/cluster/tests/test_hdbscan.py` was run with `sklearnex.preview.cluster.HDBSCAN` patched in (scikit-learn 1.9.0): **140 passed, 8 failed**, and all 8 failures are understood and deselected in scikit-learn-intelex with a recorded reason:

- `test_labelling_distinct` (4 parametrizations) and `test_dbscan_clustering` (1) -- need the `_single_linkage_tree` attribute, which oneDAL does not return (section 6.4).
- `test_hdbscan_sparse` (2) -- label numbering, see below.
- `test_hdbscan_allow_single_cluster_with_epsilon` (1) -- a real oneDAL gap, section 4.6.

Five of the eight are the one missing result option discussed in section 6.4.

The two `test_hdbscan_sparse` failures are worth a note because they look worse than they are: the test clusters the dense form (which oneDAL handles) and the sparse form (which falls back to scikit-learn) and compares the label arrays elementwise. Both find the same three clusters over the same 200 points -- the partitions are identical -- but oneDAL numbers them differently, so 66.5% of the entries differ. Nothing about the clustering is wrong; the test is simply not numbering-invariant, and it is the only test in the suite that compares an offloaded result against a fallback result.

Only one of the eight, `test_hdbscan_allow_single_cluster_with_epsilon`, is a genuine oneDAL defect. It is section 4.6.

4. Bugs found

4.1 Serial `finalize` on the shared Euclidean distance primitive (fixed)

Location: `cpp/daal/src/algorithms/service_kernel_math.h`, `EuclideanDistances::finalize`
Severity: performance, not correctness; affects every caller with a large block
Status: fixed in this PR

`finalize` turns squared L2 into real L2 by applying `max(0, .)` per entry -- floating-point round-off on the diagonal and on near-duplicate rows can push squared values slightly below zero -- and then a batched

`vSqrt` on 512-entry blocks. It did this in a plain serial loop. The function even declared a `SafeStatus` `safeStat` that it never used, which suggests a threaded implementation was intended and never written. For `bf_knn` that is harmless: it finalizes one `nTest * k` result block from inside a `threader_for`, so the outer loop provides the parallelism. For HDBSCAN's dense path the same call finalizes the entire $N \times N$ matrix -- 6.25e8 entries at 25000 rows, roughly 0.6 s on one thread while 223 threads idle.

The fix threads the sweep in 32k-entry tasks, gated on there being more than one task, so small callers keep the exact behaviour they were tuned for and do not gain a nested parallel region inside their own. Measured effect: the table in section 2.2.

While verifying this, a second latent problem turned up in the threading layer: `daal::threader_for_int64` in `cpp/daal/src/threading/threading.h` passes `threader_func<F>`, which has signature `void(int, const void *)`, where the C entry point expects `functype_int64` alias `void(int64_t, const void *)`. Any use of it fails to compile, which is why it currently has none. It is not on the HDBSCAN path and is left for a separate fix.

4.2 Unstable MST edge sort (fixed)

Location: `cpp/daal/src/algorithms/hdbscan/hdbscan_cluster_utils.h`, `sortMstEdges`
Severity: correctness -- run-to-run non-determinism
Status: fixed in this PR

The single-linkage dendrogram is built from the MST edges in ascending weight order, and `sortMstEdges` produced that order with `qSort`, which is an introsort and therefore **not stable**. Combined with the tie density described in section 3.3, equal-weight edges could be emitted in different relative orders on different runs, and the resulting labels became a function of the thread schedule.

The sort now builds an index permutation, sorts it with `std::sort` under an explicit total order on `(weight, index)`, and gathers. The comparator places NaN last deliberately: a bare `w[a] < w[b]` tie-break is **not** a strict weak ordering when NaNs are present, which is undefined behaviour in `std::sort`, not merely a wrong answer. `qSort` is retained as a fallback if any of the four scratch allocations fails.

This also aligns the CPU backend with the GPU one, which already sorts with `pr::radix_sort_indices_inplace` -- a stable sort -- so the two backends now resolve ties the same way.

A regression test was added to the Catch2 suite (`cpp/oneapi/dal/algo/hdbscan/test/batch.cpp`): three consecutive computes on a single diffuse 3000 x 2 blob, where tie density is high, must produce identical cluster counts and identical per-point labels.

4.3 Truncated MST handed to the dendrogram builder (fixed)

Location: `cpp/daal/src/algorithms/hdbscan/hdbscan_dense_batch_impl.i`
Severity: correctness -- garbage labels or out-of-bounds read
Status: fixed in this PR

Boruvka's loop exits when a round adds no edges. On a dense mutual reachability graph, which is complete, that can only happen when the matrix contains NaNs, since every comparison against a NaN is false -- i.e. on non-finite input. When it did happen, the loop fell through to `sortMstAndExtractClusters` with fewer than `nRows - 1` edges written, so the tail of `mstFrom` / `mstTo` / `mstWeights` was uninitialized and got read as tree node indices. The path now returns `ErrorIncorrectInputNumericTable` instead.

4.4 `mcsMax` bypassed for leaf clusters (fixed)

Location: `eom_select_clusters_kernel` in `cpp/oneapi/dal/algo/hdbscan/backend/gpu/kernel_impl.hpp`, and the CPU equivalent
Severity: correctness -- `max_cluster_size` silently ignored
Status: fixed (commit 24bad10fa, CPU + GPU + test)

The excess-of-mass selection applied the `max_cluster_size` bound only on the internal-node path, so a leaf cluster larger than the bound was still selected.

4.5 `clusterSelectionEpsilon` can select an ancestor and its descendant (open)

Location: `cpp/daal/src/algorithms/hdbscan/hdbscan_cluster_utils.h`,
`applyClusterSelectionEpsilon`

Severity: correctness -- produces a non-partition

Status: **open**, not fixed in this PR

scikit-learn's epsilon post-processing keeps a processed set and rebuilds the selection from scratch, which guarantees that no selected cluster is an ancestor of another selected cluster. oneDAL's version walks the selected set and adjusts in place, without that bookkeeping, so an ancestor and one of its descendants can both remain selected. The labelling step then has two valid owners for the descendant's points.

This needs the same restructuring scikit-learn uses -- collect the epsilon promotions, then rebuild the selected set with a processed marker -- and is scoped as a follow-up rather than folded into a performance PR.

4.6 No single-cluster lambda threshold in labelling (open)

Location: `cpp/daal/src/algorithms/hdbscan/hdbscan_cluster_utils.h`, `labelPoints`, and the GPU labelling kernel

Severity: correctness -- oneDAL reports no noise where scikit-learn does

Status: **open**, not fixed in this PR

When exactly one cluster is selected and `allow_single_cluster` is set, scikit-learn's `_do_labelling` does not hand the whole dataset to that cluster. It compares each sample's fall-out lambda against a threshold -- `1 / cluster_selection_epsilon` when the epsilon is non-zero, otherwise the largest lambda among the root's children -- and labels everything below it as noise:

```
elif len(clusters) == 1 and allow_single_cluster:
    parent_lambda = lambda_array[child_array == n]
    if cluster_selection_epsilon != 0.0:
        threshold = 1 / cluster_selection_epsilon
    else:
        threshold = lambda_array[parent_array == cluster].max()
    if parent_lambda >= threshold:
        label = cluster_label_map[cluster]
```

oneDAL's `labelPoints` applies no threshold at all: every point whose `pointFellFrom` resolves to a selected cluster gets that cluster's label. So on unstructured data with `allow_single_cluster=True`, oneDAL returns a single cluster containing everything, where scikit-learn returns the same cluster plus a low-density tail of noise.

This is what makes

`sklearn/cluster/tests/test_hdbscan.py::test_hdbscan_allow_single_cluster_with_epsilon` fail under patching (it is currently deselected in scikit-learn-intelex with that explanation).

The fix is small and local -- `labelPoints` already tracks `pointFellFrom[i]`, and the per-point lambda is the value on the condensed edge whose child is `i`, so the threshold is one reduction over the root's outgoing edges. It is deliberately not folded into this PR: the GPU labelling kernel needs the identical change, and shipping it on one backend only would split CPU and GPU behaviour on a parameter combination that is currently at least consistently wrong.

4.7 `kd_tree` far-child pruning is weaker than documented (open, cosmetic)

Location: nearestMrdBoruvkaQuery in `cpp/daal/src/algorithms/hdbscan/hdbscan_kd_tree_batch_impl.i`

Severity: performance, constant factor

Status: doc corrected in this PR, guard not implemented

The traversal always descends into both children and rejects the far one by evaluating the bounding-box bound at the child node, which costs the recursive call and an $O(nCols)$ bound computation. A split-plane test at the parent -- `|queryVal - splitVal| * invAlpha` raised to at least `coreQ`, which is a valid MRD lower bound for every supported Lp metric -- would reject the common case without either. It cannot prune anything the existing test would not, since the child's bounding box lies inside the half-space, so this is a constant-factor saving only. The doc comment, which previously implied a cheaper parent-side test was already in place, has been corrected.

5. Optimizations: what worked, what did not

5.1 Accepted: threaded `finalize`

Section 4.1. 1.94x on the euclidean brute-force path at 25000 x 64, 1.99x at 30000 x 3, geomean 1.54x over the brute-force set including the unaffected metrics. Labels unchanged.

5.2 Accepted: bounded select-k for core distances

Core distances need the k-th smallest entry of each matrix row. The previous implementation copied the row and called `std::nth_element` on it, so the per-thread scratch was `nThreads x nRows`. It now uses a bounded max-heap of size `minSamples + 1` (`kthSmallestBounded` in `hdbscan_distance_utils.h`), dropping the scratch to `nThreads x minSamples`.

This is a **memory** win, not a time win -- it did not move the benchmark numbers measurably, and is reported as such. At 224 threads and 100000 rows the old form reserves 179 MB of scratch for a quantity that needs 224 x 6 entries.

5.3 Rejected: Prim's algorithm instead of Boruvka (20-25% slower)

On a complete graph Prim's does strictly less total work than Boruvka: one pass over the matrix against Boruvka's $\sim \log_2 N$ rounds. It was implemented on that reasoning and measured **0.763x geomean -- 24% slower** across the brute-force set.

The reason is parallel shape, not total work. Boruvka's per-round nearest-different-component query is a single `threader_for(nRows)` over independent rows: N^2 work behind one barrier, spread across all 224 threads. Prim's is N sequential barriers with only N candidate slots to split between them, so it cannot fill a wide machine -- it reached roughly 24-way effective parallelism where Boruvka reaches 224.

The negative result is now recorded in the header comment of `hdbscan_dense_batch_impl.i` so it is not retried.

5.4 Rejected: exact nearest-MRD candidate cache (1.03x, not worth it)

Each Boruvka round asks every point for its nearest neighbour in a different component, and answering that by row scan costs one full pass over the matrix per round. Because MRD is fixed once core distances are known -- it does not depend on the component partition -- each point's k smallest MRD neighbours can be computed once and reused by every round.

This is **exactly label-preserving**, not a heuristic. Keeping the entries in ascending (`MRD, index`) order makes every uncached entry compare strictly greater than the last cached one, so the first cached entry outside the point's component is provably the row-wide (`MRD, index`) argmin -- the identical answer the

full scan's ascending strict- $\leq$ argmin gives. Only a point whose whole cached neighbourhood has joined its own component falls back to the row scan.

It was implemented, verified label-identical (cluster counts and Davies-Bouldin scores bit-identical in all 10 cases), and measured at **geomean 1.033x, median 1.026x**. Boruvka converges in very few rounds, so there are few passes to save, and building the cache costs one of them. Rejected: not worth ~100 lines in a hot kernel plus $n_{\text{Rows}} \times k \times 16$ bytes. The reasoning and the measurement are recorded in place.

6. Remaining optimization paths

Ordered by expected value.

6.1 Keep the Euclidean matrix in squared space through the MST build

This subsumes the `finalize` fix rather than building on it, and removes the sweep entirely instead of threading it.

`max` commutes with the monotone square, so `max(core_i^2, core_j^2, d^2 * invAlpha^2) = MRD^2`, and Boruvka only ever compares MRD values. The entire MST build can therefore run on the squared matrix, with the square root taken on the $N - 1$ MST edge weights at the end. That eliminates one read-plus-write pass over N^2 entries and N^2 square roots, replacing them with N of each.

Three things have to be handled, and they are the reason the matrix is un-squared today:

- `alpha` divides the pairwise distance term inside MRD, so it must become `invAlpha^2` in squared space.
- `clusterSelectionEpsilon` is an actual distance threshold on MST edge weights; applied to a squared matrix it would silently behave as `epsilon^2`. Since it is applied after the MST weights are un-squared, this is fine, but it must stay that way.
- Core distances are selected from the matrix directly. Selection is order-preserving under squaring, so the selection is unaffected, but the values must be un-squared wherever they escape the MST build.

6.2 Exploit the symmetry of the distance matrix

The matrix is symmetric and is currently computed, stored and swept in full. Halving the work is straightforward for the fill and the sweep; the row scans in Boruvka's phase 1 want full rows for contiguous access, so this is a trade between a 2x reduction in matrix work and a loss of locality in the scan, and needs measuring rather than assuming.

6.3 The `kd_tree` split-plane far-child guard

Section 4.7. A constant-factor saving on the tree path, which is already the strong path (96x-191x against scikit-learn), so this is low priority despite being easy.

6.4 Return the single-linkage tree and the membership probabilities

Not a performance item, but the largest remaining functional gap, and it blocks two scikit-learn conformance points raised in review on scikit-learn-inteleX #3124:

- `dbscan_clustering()` needs `_single_linkage_tree_`. `oneDAL` builds exactly that tree internally -- `buildDendrogramFromSortedMst` on CPU, `build_dendrogram_kernels` on GPU, already in SciPy linkage form -- but does not return it, so the method cannot work after an offloaded fit.
- `probabilities_` should hold membership strengths. The lambda values they derive from are computed inside the kernels (`buildCondensedTree` / `initClusterMetadata` on CPU,

`build_condensed_tree_kernel`'s `cond_l_ptr` on GPU), and `labelPoints` already tracks `pointFellFrom[i]`, so `probabilities_[i] = min(lambda_i, max_lambda[c]) / max_lambda[c]` is a small addition on top of a new `result_options::probabilities`.

Both are additive `result_options` on an existing internal quantity, which makes them cheap relative to their conformance value.

7. Test coverage added

In `cpp/oneapi/dal/algorithm/hdbscan/test/batch.cpp`:

- **Cross-method agreement at scale.** 4 x 1000 points in 3 dimensions, separation 20.0, spread 1.0, `min_cluster_size=25`, with `min_samples` generated over `{5, 50}`. Asserts that brute force finds the 4 planted clusters, that all three methods agree on the cluster count, and that `brute` vs `kd_tree` and `brute` vs `ball_tree` induce the same partition. The existing suite covered only small inputs, where the threaded MST paths are not exercised.
- **Run-to-run stability.** The regression test for section 4.2: a single diffuse 3000 x 2 blob, where tie density is high, computed three times with `descriptor(15, 5)`; cluster counts and per-point labels must be identical.

Both use a small deterministic LCG blob generator rather than an external dataset, so they run under the default Catch2 filters. All four bazel targets pass: `test_batch_host`, `test_batch_dpc`, `test_badarg_host`, `test_badarg_dpc`.